Research report and reproducible artifacts

Toward a smaller PA-consistency machine

An audit of the five U208 proposals, a verified source specialization, and new reduction experiments

Prepared for Vladimir Reshetnikov

29 October 2026

Result status. This investigation did not obtain a validated Turing machine with fewer than 208 working states. It did establish a new, executable-checker-backed reduction of the inherited register source from 3,909 to 3,904 primitive occurrences, preserving halting from its canonical initial state. The five submitted U208 packages reduce to two transition-graph classes. Their advertised 209-to-208 savings implement the same fold and cannot be added together.

Included: this report in LaTeX and PDF; exact comparison inputs and isomorphism witnesses; the optimized source and its inductive invariant checker; reproducible failed 207-state experiments; search code, receipts, and pinned upstream reference archives.

Proof status: ordinary mathematical arguments and finite executable certificates. No new end-to-end MM0, Rocq, or Lean certificate is claimed.

Contents

1 Results and scope **2**

1.1 What was inspected and what was executed 2

2 Comparison of the five proposals **3**

2.1 Nine literal tables, two graph classes 3

2.2 The source programs agree beyond their addresses 3

2.3 The common 209-to-208 fold 4

3 The logical endpoint: unary U and the machine's actual calculus **5**

3.1 The precise unary restriction 5

3.2 The upper interpretation and proof transport 5

3.3 The 208-state source does not enumerate the separate unary stream 6

4 Literal runtime verification and its scope **6**

5 New result: a 3,904-primitive source **7**

5.1 Semantics and a precise theorem 7

5.2 A sound zero/positive register abstraction 8

5.3 First substitution: remove a zero-input transfer loop 8

5.4 A three-category abstraction of one register 9

5.5 Second substitution: remove a redundant target-test cleanup 9

5.6 The exact control-flow projection 9

5.7 What this optimization does and does not save 10

6 New tape-context invariants and reduction searches **11**

6.1 An unbounded-history finite-window invariant 11

6.2 Search families and bounded negative conclusions 11

7 Why two apparent 207-state reductions fail **12**

7.1 Removing one state from the origin detector 12

7.2 Reassociating the source and merging 85 with 154 13

8 Research implications and the next concrete problems **13**

A Artifact guide and reproduction **14**

B Identity of the new source artifacts **15**

1 Results and scope

The target is a binary, one-tape Turing machine M, started on the all-zero tape, for which

$$\mathsf{PA} \vdash \neg\,\mathrm{Halt}(M) \longleftrightarrow \mathrm{Con}(\mathsf{PA}),$$

with fewer than 208 working states. A separate halt label is not counted as a working state. A smaller numerical source, a shorter execution on sampled inputs, and a smaller literal transition table are different achievements. This report keeps those distinctions explicit.

The requested state-count target remains open in this investigation. The positive new result is a source-level halting-equivalence theorem with a compact, independently replayable finite argument. Two sound abstractions of the nine-register program prove that three primitive occurrences are unreachable and that two tests are identity steps whenever reached. Removing them gives 3,904 primitives and 5,076 nonhalting ordinal edges, instead of 3,909 and 5,083. The initial source configuration and the retained halt instruction are preserved.

This is useful but does not itself yield a smaller Turing machine. Under direct hash-consing, the number of distinct syntax subtrees increases from 197 to 200: removing occurrences can destroy sharing elsewhere. The literal machine also shares decoder states with arithmetic routines. Any physical reallocation must preserve both uses. These two facts explain why a genuine source simplification has not yet become a state deletion.

The input comparison produces a second concrete result. The five U208 archives contain nine designated final transition tables, including alternatives. All nine belong to exactly two rooted, action-preserving graph-isomorphism classes. Both classes implement the same sequence of 3,909 source primitives and the same ordinal control-flow graph. The five advertised final savings are orientations or relabelings of the same contextual identification of native states 172 and 211.

The research extended the supplied searches in several directions: unbounded-history finite-window invariants; head-offset and tape-tag relations; state folds checked at complete interpreter returns; shared selector synthesis; a source-reachable register abstraction; scanner repair; and repairs of a tempting source-reassociation fold. None of the completed searches supplied an accepted sub-208 table. They exclude only their stated finite search families, not all machines of a given size.

1.1 What was inspected and what was executed

The ProveIt inputs were fixed at commit `c086b5adaa62280093657689 46d691dfc49a20bb`, under `docs/incoming`. Six ZIPs were downloaded: the five U208 proposals and `pa_unary_power_cover.zip`. Their bytes were checked against both SHA-256 digests and the Git blob identities advertised by the pinned directory listing. The exact retrieval manifest and archive digests accompany this report.

The review covered all five top-level reports, all nine designated final tables, the complete unary lower-interpretation and upper proof-transport arguments, the principal local reduction checkers, and the inherited native-calculus calibration. Every package's quick or finite replay was freshly run successfully. A separate fresh full symbolic interpreter replay also passed for both graph classes: 5,084 contracts and all 416 rows in each class. Existing upstream receipts are distinguished from these newly executed checks.

The contextual references to M254, Theory K, and K215 establish the construction lineage and the desired proof standard. In particular, the K215 repository states an MM0 theorem in PA for its exact literal machine and ZFC inconsistency [4]. That theorem does not automatically certify a changed axiom emitter, a changed source program, or one of the U208 tables. The present work does not recheck the large K215 proof object.

2 Comparison of the five proposals

2.1 Nine literal tables, two graph classes

Call the two final graph classes A and B. The comparison fixes the start state, halt label, read symbols, write symbols, and head directions. It permits state renaming only; it does not identify tape reflections or symbol complements.

Proposal	Main	Other	Main contribution of the presentation
U208_ConPA	A	—	Three-row orientation, independent local checks, and analysis of failed source compression.
Conditional Context	A	B	A particularly clear inductive tape-context invariant and two-row orientation.
Entry Context	A	B	Reproducible bounded-context discovery and separation of proof obligations.
History	A	B	Plain-native orientation, portable fold checker, and an explicit failed-rotation witness.
u208_research	B	A	The Compatibility layout as principal output and symbolic-runtime receipts for both classes.

Table 1: All five archive names end in `2026-10-29.zip`. Full paths and exact bytes are retained in the provenance manifest.

Conditional Context and Entry Context have byte-identical principal native and dense tables; their alternatives are also byte-identical. Within the other matching classes, serialization and state labels vary. The differences do not define independent halting algorithms.

Every designated table has 208 working states, 416 transition rows, and all 208 working states are reachable in its transition graph from the start. This is graph reachability, not a claim that every state occurs on the blank-tape computation.

The independent comparison uses breadth-first canonical numbering from state 0, exploring read-zero before read-one. It emits each state's two write/direction/target triples, treating the halt as a distinguished endpoint. Equal serializations give an explicit rooted action-preserving isomorphism. The complete numbering maps are saved, so the argument is not merely a comparison of hashes. The canonical SHA-256 fingerprints are:

```
A: 5d858b5d7528dd98c5272f83d308c782ae323a47fc394a73ad9efda1096a5284
B: f0c6098fabd68fce3fca0c6ac78a1b4e9dbb4ded60437b84d3e9d9ccda74f11d
```

These digests depend on the stated serialization; the saved bijections are the substantive witnesses.

2.2 The source programs agree beyond their addresses

An independent source-tree resolver reconstructs the control-flow graph from subtree intervals and ancestor walks. The A and B sources have identical ordered opcode/operand pairs and identical ordinal successors. The primitive counts are:

CI	CD	CB	CC	CH	Total
1558	1175	80	1095	1	3909

There are 5,083 nonhalting ordinal edges. Splitting each decrement into its zero and positive cases, and including the halt contract, gives 5,084 primitive contracts.

The binary addresses differ at 720 primitive occurrences. Class A has maximum address depth 42, class B depth 41; both have maximum consecutive-zero run six. B comes from an earlier reassociation of an eight-factor source region and a different choice in a preceding physical-state allocation. The 172/211 fold is then applied to both. A depth reduction alone does not prove a global execution-time improvement.

Both classes should be retained as inputs to further searches. Conditional/Entry offers a clear class-A proof interface; the compact research package is convenient for replaying both classes. Their distinct layouts can expose different sharing opportunities even though their abstract programs agree.

2.3 The common 209-to-208 fold

The following uses the Joint native labels in the Conditional presentation. Label 254 is the halt. Delete 172, retain 211, and change exactly these retained rows:

Row	Parent	Child
116/0	$0\,R\,172$	$0\,R\,211$
211/0	$0\,R\,157$	$0\,L\,217$

Let $f(172) = 211$ and let f fix every other label. If t_j denotes the bit at offset j from the head, the parent has the following inductive invariant:

$$q = 116 \Rightarrow t_{-1} = 1, \qquad q = 172 \Rightarrow (t_{-2}, t_{-1}) = (1, 0),$$
$$q = 217 \Rightarrow t_{-1} = 1, \qquad q = 163 \Rightarrow t_{-1} = 1 \vee t_{-2} = 1,$$
$$q = 211 \Rightarrow t_0 = 1 \vee t_{-1} = 1.$$

It holds initially because the start is none of these states. The complete incoming-edge census proves preservation: entries to 116 and 217 write one and move right; 116/0 establishes the two-cell word before 172; the entries to 163 establish its disjunction; and its leftward zero or rightward one exit establishes the condition at 211. The census includes physical arithmetic entrances, not just source-decoder entrances.

Four literal identities suffice for the exceptional rows:

Old state	Read	Child steps	Required extra bits	Child endpoint
172	0	7	$t_{-2} = 1, t_{-1} = 0$	$(67, h-1, T)$
172	1	1	none	$(184, h+1, T)$
211	0	3	$t_{-1} = 1$	$(157, h+1, T)$
211	1	1	none	$(184, h+1, T)$

For the seven-step case the states after each step are

$$217,\ 163,\ 211,\ 217,\ 215,\ 157,\ 67,$$

and the head offsets are $-1, 0, -1, -2, -1, 0, -1$. Only the three specified cells are read and every write restores the bit already there. The other zero case uses three steps. All 414 ordinary parent rows commute with f.

Induction on parent time gives a positive-block simulation with the same complete tape and head at boundaries. If the parent halts after T_{209} steps, then

$$T_{208} = T_{209} + 6N_{172,0} + 2N_{211,0}, \qquad T_{209} \le T_{208} \le 7T_{209},$$

where the occurrence counts are taken along the parent run. The positive block lengths and absence of an internal halt also prove nonhalting preservation. This works from state 0 on every common initial tape. No consistency assumption is used.

Keeping 172 instead of 211 gives the other orientation. Plain/Compatibility use the corresponding helper label 214. These are the same saving, so combining the five descriptions does not produce a 207-state machine.

3 The logical endpoint: unary U and the machine's actual calculus

3.1 The precise unary restriction

Theory U uses equality-free classical first-order logic, a nonempty domain, and one binary relation E. Write

$$\mathrm{Sub}(x,a) := \forall t\,(E(t,x) \to E(t,a)).$$

For each formula $\varphi(x)$ with no other free variable, the schema is

$$\forall a\,\exists b\,[\forall x\,(E(x,b) \to \varphi(x)) \;\wedge\; \forall x\,(\mathrm{Sub}(x,a) \to (\varphi(x) \to E(x,b)))]\,. \tag{1}$$

Arbitrarily many bound variables and arbitrary quantifier complexity are permitted. The source a is not allowed freely in the matrix. The output is a weak cover: its elements must satisfy the matrix, and it must contain every matrix-satisfying subset of a. It may contain further objects satisfying the matrix. Substituting $\mathrm{Sub}(x,a) \wedge \varphi(x)$ for the matrix without binding a would violate the unary condition.

The supplied lower interpretation is technically substantial. Its reviewed dependency order matters. Coextensionality and saturated numerical representatives are defined first. Empty representatives and local predecessors do not presuppose total successors. A failure-filter argument proves induction before successor totality: if a proposed counterexample has no earlier failure inside its member profile, the counterexample itself is the first failure. This avoids assuming its successor.

Parameters are internalized by binding them in a unary formula of the form "for every parameter tuple, base and step imply the property at this representative." Comparison and least failing numerical tuples then supply bounded numerical comprehension. The support object is selected outside the unary matrix; the least tuple is quantified inside it. Saturated successors, powers, pair codes, and bounded graph comprehension follow. Canonical history formulas existentially quantify earlier histories instead of placing an arbitrary old graph freely in a filter. History uniqueness is established before extension, and a numerical value bound is proved separately. Addition and multiplication follow by iteration, with induction already available.

No concrete circularity or illicit matrix parameter was found in this review. That is a reviewed mathematical argument, not a newly kernel-checked interpretation. The full input manuscript is included unchanged in the reference archive so that these dependencies can be inspected directly.

3.2 The upper interpretation and proof transport

In PA interpret $E(x,a)$ as "the xth binary bit of a is one." PA proves $E(x,a) \Rightarrow x < a$, and $\mathrm{Sub}(x,a) \Rightarrow x \le a$. For each fixed translated matrix, induction forms the finite bitset of exactly those $x \le a$ satisfying the matrix and the subset condition. This bitset satisfies both clauses of (1). The argument is formula-by-formula; it does not postulate a uniform arithmetic truth predicate.

Let W allow arbitrary additional parameters, including the source, in the same weak-cover schema, and let IR be $\forall x \neg E(x,x)$. The same PA interpretation validates $\mathsf{W}+\mathsf{IR}$, while W contains all unary

instances. The supplied finite-refutation translations give the consistency cycle

$$\mathrm{Con}(\mathsf{PA}) \implies \mathrm{Con}(\mathsf{W}+\mathsf{IR}) \implies \mathrm{Con}(\mathsf{U}) \implies \mathrm{Con}(\mathsf{PA}).$$

The translations use structural formula recursion, fresh variable selection, and fixed derivation templates. Their stated correctness is provable by PA induction on finite syntax. The manuscript provides this uniformity mathematically, but does not supply an implemented translator for arbitrary input proofs or a new exported PA certificate.

3.3 The 208-state source does not enumerate the separate unary stream

The final tables implement the inherited native Cantor-coded calculus for $\mathsf{W}+\mathsf{IR}$. They do not implement the unary archive's separate positive dyadic proof stream. The distinction includes the target convention: native truth is code 0 and the contradiction target is code 1, whereas the separate dyadic stream targets 0.

The native constructors use

$$P(x,y) = \frac{(x+y)(x+y+1)}{2} + y,$$

$$I(x,y) = 4P(x,y), \quad R(i,j) = 4P(i,j)+1, \quad Q(i,f) = 4P(i,f)+2, \quad E_{=}(i,j) = 4P(i,j)+3.$$

The four constructors represent implication, membership, universal quantification, and auxiliary equality. Code 0 is a deliberate truth alias; code 1 can be decoded as $E(v_0, v_0)$ and normalized to falsity using irreflexivity. Treating 1 as false without that justification would be a soundness error.

The retained axiom emitter uses the matrix code itself as a fresh output-variable label. For a positive code, every variable label occurring in its decoding is smaller than the code. The low codes 0 and 1 require separate collision calculations. The existing calibration also handles equality via membership indiscernibility, vacuous binders, source/output collisions, proof-description padding, and cofinally many successful proof descriptions.

The literal blank initializer enters an interior axiom-generator suffix. The source stack therefore contains an opaque tail before it becomes a clean proof-search state. Nine generated theorem heads and the scheduler's credit argument keep that tail from being exposed. Because $P(0,0) = 0$, a separate logical stack-length parameter is needed even though the numeric stack representation can collapse a zero pair. A finite startup trace alone does not prove this unbounded protection.

These obligations are separate from the 172/211 state fold. The fold proof transfers an already justified parent halting predicate; it cannot repair a flaw in the calculus or theory interpretation. This is also why the K215 formal theorem cannot simply be retargeted by replacing the words "ZFC" with "PA."

4 Literal runtime verification and its scope

The source has nine logical registers. The logical-to-physical order is

$$(6, 2, 0, 4, 3, 7, 8, 5, 1).$$

A canonical tape frame has a left sentinel, a binary program-counter word, a zero gap, a header, nine blocks $B(v) = 0\,1^{v+1}$, and a nonempty finite list of spare blocks. Registers and the gap may be arbitrarily large. A positive decrement changes the gap; an increment appends a spare; overflow of the counter may shift the frame origin. A universal interpreter contract must account for all of this, including the complete exterior tape.

The fresh quick checks bind the literal tables to their source, reconstruct the folds, check exact row counts and renumberings, and replay startup. The common startup receipt has:

Quantity	Value
Literal transitions from blank	48,140
Native dispatcher state and head	$84, -2$
Header coordinate	54
Source ordinal	136
Program-counter word	`001100110101010`
Logical registers	$(1, 0, 13, 0, 0, 1, 0, 0, 1)$
Spare blocks	43, each representing 1
One cells	120

An additional 141-step source bridge connects the specified interior entry at ordinal 106 to that source seed. Neither trace alone establishes the consistency endpoint; they establish the literal startup boundary used by the unbounded arguments.

The inherited symbolic interpreter engine proves reusable arithmetic blocks and then all 5,084 guarded source contracts, covering all 416 physical rows. Its supplied inference counts are 1,251,865 for A and 1,250,564 for B. These are checker-inference counts, not full consistency-search execution times.

Fresh full replay. A new invocation of the compact research package's `verify.py--full` passed for both final graph classes. Class B checked 5,084 contracts, 416 rows, and 1,250,564 symbolic inference applications. Its class-A alternative checked 5,084 contracts, 416 rows, and 1,251,865 applications. The run also verified 167 retained dependency member hashes and the nested theory-U binding. Elapsed time in the preparation environment was approximately 187 seconds. The complete new receipt and console log are `audit/replays/research_full.json` and `research_full.log`. These counts were freshly reproduced, not copied only from the supplied receipts.

The result remains an executable symbolic-induction argument, not a proof-assistant kernel certificate. In particular, an ordinary Python checker and its arithmetic lemmas are part of this artifact's trust basis. The evidence is stronger than a finite selection of register values, but it should not be described as an MM0 or Rocq replay.

5 New result: a 3,904-primitive source

5.1 Semantics and a precise theorem

`CI` j increments register j and advances. `CD` j takes a zero edge without changing the register, or decrements a positive register and takes its positive edge. `CB` k and `CC` k are register-preserving jumps determined by the binary source tree. `CH` halts. All ordinal indices below are zero-based indices in the original 3,909-leaf class-A source.

Let S be that source, and let S' be the transformed source delivered in `source_optimization/generated/optimized_sites.json`. Both use the same primitive semantics. Write

$$c_* = (136; 1, 0, 13, 0, 0, 1, 0, 0, 1).$$

Theorem 5.1 (Source specialization)**.** *The executions of S and S' from c_* halt together. At matched retained source locations they have identical register vectors. Each step of S' is simulated by one or two steps of S. The new source has 3,904 primitives and 5,076 nonhalting ordinal edges.*

The proof below is independent of any consistency assumption. The finite invariant calculations

and the induction on coded runs are formalizable in PA. The delivered checker executes the finite premises; it does not output a formal PA derivation.

5.2 A sound zero/positive register abstraction

For a concrete register vector $\mathbf{r}$, define

$$\alpha(\mathbf{r}) = \sum_{j=0}^{8} 2^j [r_j > 0].$$

There are only 512 possible masks. At each original program location p, compute a set I_p of masks, starting at $(136, 293)$. Abstract increments set their bit. A decrement with a zero bit follows only its zero edge. A decrement with a positive bit follows its positive edge and permits both possible resulting masks: the bit can remain positive or become zero. Jumps preserve the mask; halt has no successor.

Lemma 5.2 (Soundness of the mask invariant)**.** *If the family (I_p) contains the initial mask and is closed under every abstract successor, then every concrete reachable configuration $(p, \mathbf{r})$ satisfies $\alpha(\mathbf{r}) \in I_p$.*

Proof. The statement holds at the seed. Each concrete instruction effect is included in the corresponding abstract transition rule. Induction on concrete execution length therefore preserves membership. The only nondeterministic abstraction is positive decrement, and its two alternatives cover exactly the possibilities $1 \mapsto 0$ and $n + 1 \mapsto n > 0$. □

The standalone checker discovers the least reachable family and then separately checks initial inclusion and every abstract closure obligation. The family has 278,384 location/mask pairs. At ordinal 3847 it is exactly

$$I_{3847} = \{1, 5, 17, 21, 257, 261, 273, 277\}.$$

Bit 1 is zero throughout this set. The sets at 3848 and 3849 are empty.

A smaller final certificate. The discovery used the joint mask, but the first substitution needs only a zero/positive abstraction of register 1. Admit both branches of every decrement on a different register; use the ordinary two-category rules for register 1 itself. This family contains only 6,430 location/category pairs. It again assigns zero alone to 3847 and nothing to 3848 or 3849. Its initial inclusion and every successor are checked separately. Together with the register-6 certificate below, the final source proof therefore needs only two small single-register invariants. The larger joint-mask certificate is retained for comparison and for source-restricted machine searches.

5.3 First substitution: remove a zero-input transfer loop

The subtree rooted at `0011011100` is:

Ordinal	Address suffix	Instruction	Successors
3847	`0`	`CD 1`	zero:3850; positive:3848
3848	`10`	`CI 6`	3849
3849	`11`	`CC 2`	3847

It transfers the contents of register 1 to register 6. By the proved mask invariant, its input register is always zero at reachable entry. It immediately exits to 3850 and changes no register.

The premise also has a direct local explanation. The sole external entrance into the preceding comparison region at 3837 is the zero exit of `CD` 1 at 3834. The instructions 3837–3846 inspect and change only registers 5 and 6. They therefore preserve register 1 at zero before entering 3847. Its positive exit is impossible, so the hypothetical back edge from 3849 cannot create a new entry. The complete incoming inventory and the initial location 136 justify this invariant by induction.

The parent at `001101110` is a sequence of this loop and a closed remainder. Replace the parent by its right child, formerly rooted at `0011011101`. This removes all three occurrences. The two body instructions are unreachable; the test is an identity step whenever reached.

5.4 A three-category abstraction of one register

The second premise needs only register $X = r_6$, tracked in categories 0, 1, and ≥ 2. All branches of decrements of other registers are admitted nondeterministically. An increment of X maps 0 to 1 and maps either positive category into ≥ 2. A decrement of X maps 1 to 0 and permits both 1 and ≥ 2 from the ≥ 2 category. Zero has only its zero branch. Jumps and instructions on other registers preserve the category.

The same closure-and-induction argument proves soundness. The discovered family has 9,651 location/category pairs. It gives category set $\{0\}$ at 3891 and the empty set at 3892. This is substantially smaller than tracking all nine registers in a three-category Cartesian product.

There is also a short local explanation. Put $H = r_5$. The relevant complete control-flow region is:

Ordinal	Instruction	Successors	Ordinal	Instruction	Successors
3880	`CD` X	3883/3881	3888	`CD` X	3894/3889
3881	`CI` H	3882	3889	`CD` H	3891/3890
3882	`CC` 2	3880	3890	`CC` 1	3889
3883	`CI` X	3884	3891	`CD` X	3893/3892
3884	`CD` H	3888/3885	3892	`CC` 2	3889
3885	`CD` X	3887/3886	3893	`CB` 1	3895
3886	`CC` 2	3884	3894	`CH`	halt
3887	`CB` 1	3889			

For decrements the listed successors are zero/positive. The complete graph has no additional entrances into 3883–3894. Entry to 3883 is a zero exit of 3880, so $X = 0$; its increment establishes $X = 1$. The comparison region maintains $X \leq 1$. Any decrement of a positive X there leaves zero; a zero exit already has zero. Consequently every entry to 3889 has $X = 0$. The 3889–3890 loop changes only H, so 3891 also has $X = 0$.

5.5 Second substitution: remove a redundant target-test cleanup

The subtree at `1110011` is

$$((\texttt{CD}\ 6, \texttt{CC}\ 2), \texttt{CB}\ 1),$$

with old leaf ordinals 3891, 3892, 3893. Its decrement always follows the zero edge to the final `CB` 1. Replace the subtree by that single branch at `1110011`. The old and new branch both leave for the first leaf under `1111`, old ordinal 3895. Two occurrences disappear, while all register values are unchanged.

5.6 The exact control-flow projection

Delete

$$D = \{3847, 3848, 3849, 3891, 3892\}.$$

Let f be the increasing bijection from retained old ordinals to $\{0, \ldots, 3903\}$. Extend it where needed by

$$f(3847) = f(3850), \qquad f(3891) = f(3893).$$

The remaining deleted locations are unreachable by the proved invariants.

The checker reconstructs each source tree and resolves its graph twice: once by subtree intervals and outward ancestor walking, and once by binary increment, deletion of designated zero/one ancestors, and zero-padded prefix decoding. The latter also checks that no stray one lies beyond the decoded leaf. The two resolvers agree on every old and every new instruction.

The finite projection check then verifies that every retained instruction has the same opcode and operand, and every outgoing edge has exactly its projected successor. It verifies the two identity skips, retention of the unique halt, and preservation of the initial ordinal, address, and instruction.

Proof of the source-specialization theorem. At a matched retained location, the old and new instruction have identical effects on registers. The projected target is either the next retained old location or the result of one proved identity test. Thus one new step matches one or two old steps. No skipped nonidentity instruction can occur: 3848, 3849, and 3892 are excluded by the inductive invariants. The two other deleted instructions always take their zero branch whenever reached and cannot form a deleted cycle.

Inductively the old and new computations have identical registers at every matched boundary. The halt is retained and each block has positive length, proving both directions of halting equivalence, including the case of an infinite computation. The leaf and edge counts are the checked finite counts after the substitutions. □

5.7 What this optimization does and does not save

Metric	Original	Specialized
Primitive occurrences	3909	3904
Nonhalting ordinal edges	5083	5076
Distinct syntax subtrees	197	200
Maximum source address length	42	42
Maximum consecutive-zero run	6	6
Literal Turing-machine working states	208	not allocated

The increased subtree count is not a contradiction: the removed transfer and comparison patterns were shared at other occurrences. Specializing only these occurrences can create new ancestors. Moreover, a syntax subtree is not a physical-state allocation certificate. A smaller machine will require a successful reassociation or a guarded allocation exploiting the proven reachable contexts, followed by complete runtime and initializer verification.

A further source-only reassociation reached 199 distinct subtrees, but increased maximum address depth to 53 and maximum zero run to 11. This worsens the origin-delimiter requirement and is not a better physical allocation baseline. It is excluded from the primary optimized-source identity above.

6 New tape-context invariants and reduction searches

6.1 An unbounded-history finite-window invariant

For a radius r, abstract a concrete Turing-machine configuration by its state and the $2r + 1$ bits surrounding its head. A transition writes the current bit and shifts the window. The newly exposed boundary bit is allowed to be either 0 or 1. Start with every window in state 0 and iterate to a fixed point.

Proposition 6.1 (Window soundness)**.** *Every concrete configuration reachable from state 0 on every initial binary tape projects into this abstract reachable set.*

Proof. All initial windows are included. A concrete successor has the written and shifted bits prescribed by the abstraction; its single newly exposed bit is one of the two admitted choices. Induction on the run length proves inclusion. No finite history-depth restriction occurs in this argument. □

Class	Radius	Possible pairs	Reachable pairs	DNF cubes
A	7	6,815,744	961,765	2,692
A	9	109,051,904	11,318,840	7,796
B	7	6,815,744	967,781	2,698

The radius-seven A certificate was independently checked for initial inclusion, both abstract successors, and equality with its decision-diagram path cover. This involved 1,923,274 successor checks. The finite set is a proved overapproximation, so impossible concrete windows may remain. A failed fold against such a window need not refute a fold restricted to a smaller concrete invariant.

6.2 Search families and bounded negative conclusions

The delivered notes specify exact drivers and receipts. The following summarizes the main completed families. “No survivor” means no candidate passed the stated checks within that family; it is not a state-count lower bound.

Family	Scope	Outcome and limitation
Direct local aliases	Same-head identities on depth-eight and fixed-point contexts; blocks up to 45 steps in one variant.	No exact survivor on either class. A different simulation relation or longer essential macro is not excluded.
Two-row representative synthesis	A: 36,412,128 row candidates; B: 36,190,224. Positive blocks at both endpoints, each bounded by 19 steps.	2,454 A and 1,396 B sampled composites all failed exact context checks. Separate synthesis can miss coupled row changes.
Head-offset and tape-tag relations	A: 36,639 offset aliases, 60,674,184 offset row mutations, and 73,278 tag aliases.	No exact survivor. These cover specified offsets and local bit complements only.
Direct aliases at complete primitive returns	42,642 ordered pairs, excluding deletion of native 0 or 84 in this driver.	23 targeted survivors; all failed the full 15,252-case finite suite.

Family	Scope	Outcome and limitation
One-row folds at complete primitive returns	468 compatible ordered state/read pairs; 389,376 modifications per graph class, including halt targets.	26 targeted A survivors and 32 B survivors; all failed the full finite suite. Deleting 0 or 84 was allowed here.
Relocation of the halt	343,616 patches combining the 68/196 fold with one retained row changed to halt.	No survivor on selected complete primitive tests. Halting head and tape were unrestricted.
Two-row scanner repair	828 single patches and 5,832,432 paired patches after deleting native 34.	No exact return on all 5,093 collected scanner entries, with a 600-step cap per entry.
Shared-selector synthesis	Two context folds sharing one physical selector, allowing arbitrary replacement writes, moves, and targets.	No survivor of the bounded local screen. The screen does not cover every interacting multi-state synthesis.
Source-reachable local contexts	278,384 mask states; 835,152 finite frames; 393,296,588 literal transitions.	No one-row local fold or complete edge-by-edge bypass. Abstract masks and positive-value samples do not specify the exact blank run.
Source-reachable complete endpoints	71,622 cases selected from permitted masks and register-6 categories; 389,376 one-row mutations.	48 targeted survivors, none passing the full selected corpus. Magnitudes remain sampled; this is not a universal contract.
Repair of the 85/154 rotation fold	89,748 one-row and 516,051 heuristic two-row mutation evaluations.	No full 5,084-case finite survivor. Second-row selection was restricted to collected footprints.

The finite primitive suite uses three register-value assignments per guarded source contract, for 15,252 cases. It is an efficient rejection filter, not a universal proof. Complete-endpoint checking was useful because it removed the requirement that a changed physical row simulate one old row in a short local block: a candidate was allowed to take a different path before reaching the same complete source-level boundary.

The source-reachable mask corpus is a different filter. It removes impossible zero/positive combinations but samples positive magnitudes. Its local-window search was still negative. A second global-endpoint screen used 71,622 cases selected to exercise feasible source locations and branches, and was also negative. Simply filtering the old 15,252 cases by the invariant was inadequate: many source routines then went unexercised and thousands of irrelevant mutations survived. This is why the source-restricted corpus was generated afresh. Additional alternate-layout experiments are recorded separately in the research notes; no unrecorded search is included in the counts above.

7 Why two apparent 207-state reductions fail

7.1 Removing one state from the origin detector

The tempting patch deletes native state 34 and changes 23/0 from $0\,L\,34$ to $0\,L\,75$. Static source words suggest a shorter zero-run threshold might suffice. The transient interpreter tape refutes this reasoning.

Take the first primitive, `CD` 8 at address `000000`, with all nine registers zero, one zero spare, and gap zero. At an internal return to the origin scanner, seven zero cells lie between the anchor at -1

and a temporary one block at cells 7 through 56. The old execution reaches native 23 with head 1 and then follows

$$(23, 1) \longrightarrow (34, 0) \longrightarrow (75, -1) \longrightarrow (84, 0).$$

The modified execution follows

$$(23, 1) \longrightarrow (75, 0) \longrightarrow \cdots \longrightarrow (75, 7) \longrightarrow (84, 8).$$

The complete tape agrees, but the dispatcher head is displaced by eight cells. A source-code zero-run bound that ignores temporary runtime counter forms is insufficient. The delivered counterexample records the exact entry tape and endpoints.

This witness rejects a universal primitive-interpreter contract. It does not by itself prove that this bad configuration occurs on the actual blank-tape run. Accordingly, the correct conclusion is that the proposed proof route fails and no validated candidate results, not that a consistency search was observed to give an incorrect global answer.

7.2 Reassociating the source and merging 85 with 154

The source rotations

$$((98, 12), 150) \mapsto (98, (12, 150)), \qquad ((166, 12), 150) \mapsto (166, (12, 150))$$

can make two decoder roles coincide. A subsequent identification of 85 and 154 gives 207 state labels and preserves the primitive sequence and ordinal source graph. It nevertheless changes arithmetic paths that enter those same physical states backwards, outside ordinary source decoding.

A freshly rebuilt version using the Three-cell native labeling fails the finite gate at old ordinal 25, a zero-case `CD` 1: it does not reach the expected complete canonical frame within 100,000 literal steps. A single-row repair changing 152/0 to $0\,L\,133$ fixes earlier finite cases, but fails the same gate at ordinal 70, a *positive*-case `CD` 5 with register 5 equal to one. The first dispatcher visits have stray program-counter bits and are not canonical source returns. Interpreting a decoded prefix at that intermediate visit as a completed primitive would be incorrect.

The portable replay records both bounded failures with exact tapes. It does not prove that the wrong table never eventually reaches the required frame, nor that its blank-input halting behavior differs. The History archive contains its own distinct failed-rotation witness; the mutated tables and their diagnostics must not be conflated. The fresh bounded repair search tested 89,748 one-row and 516,051 two-row mutation evaluations without finding a full finite survivor. The two-row screen selected possible second rows from execution footprints, so it is not exhaustive over arbitrary pairs of changed rows.

The failed arithmetic entries are precisely why source equivalence is insufficient for this physical compiler. A safe allocator needs to preserve every relevant physical entry contract, or prove a new restricted runtime invariant that justifies changing it. An equality of decoder subtrees alone provides neither.

8 Research implications and the next concrete problems

The strongest new positive artifact is the source-specialization theorem. It narrows a specific allocation problem: implement the two justified occurrence-specific simplifications while retaining or improving physical decoder sharing. A useful next attempt is to reassociate the closed factors surrounding the substitutions, optimize the number of distinct subtrees, and then allocate against the complete arithmetic-entry constraints. A source-only optimum should be treated as an input to that allocator, never as a state-count certificate.

The two retained source invariants also offer a sound way to specialize interpreter obligations. A future global fold may fail on an arbitrary canonical frame but succeed on every frame reachable from c_*. Such a proposal needs a universal contract over the certified mask/category family, not just the sampled positive register values. The invariant must be transported through the literal startup and through the modified simulation. This is a concrete extension of the present source proof, rather than an assumption about the inaccessible eventual blank computation.

Changing the machine-facing theory remains legitimate. The current unrestricted cover presentation avoids checking the unary free-variable restriction, while the PA bit interpretation and inclusion of unary U maintain the needed consistency strength. The separate positive dyadic emitter could also be compiled, but its formulas, tags, truth convention, and proof enumerator differ from the retained Cantor implementation. A pairing-count saving in that abstract emitter is not yet a physical state saving. Any comparison must include the cost of the new interpreter and initializer.

The final target requires all of the following in one artifact: an explicit table with fewer than 208 working states; a sound literal halting-equivalence or direct proof-search theorem; exact blank initialization; and a PA-formalizable connection to Con(PA). The present archive supplies the audited 208-state baseline, a new verified source simplification, and reproducible evidence identifying several unsuccessful routes. It does not claim the missing smaller table.

A Artifact guide and reproduction

The archive is organized around independently checkable claims:

`audit/` Nine exact final dense tables, the two source maps, provenance, independent graph/source comparisons, and fresh replay receipts.
`source_optimization/` Original source, specialization checker, mathematical proof notes, optimized source, graph projection, and finite inductive certificates.
`context_research/` Curated context-invariant and search code, exact search-scope notes, and the rejected scanner experiment.
`rotation_repair/` Rebuilt rejected 207-state tables and portable counterexample replay.
`additional_search/` Shared-selector and source-context experiments with their recorded limits.
`references/` The unchanged compact two-class U208 package and the unchanged unary-U package, including their calibration dependencies and notices.

The root `reproduce.py` performs the fast comparison and source-specialization checks and replays the small negative witnesses. Optional commands regenerate larger invariants, rerun broader searches, and invoke the original full symbolic engine. Exact commands, dependencies, and expected outputs are given in `README.txt`. All paths are relative to the unpacked archive; no network access is needed for the default replay.

The source-specialization checker uses only Python's standard library. It explicitly raises exceptions for failed obligations, so `python3-O` does not disable its checks. The source calculation was freshly replayed both normally and with optimization, producing identical generated files. A second standard-library-only radius-seven verifier is included in the curated capsule; the initial exploratory verifier used NumPy. Search executables require a C++17 compiler. The reference symbolic interpreter deliberately requires assertions enabled.

To rebuild this report, use a standard LaTeX distribution with the packages listed in its preamble and run `pdflatexreport.tex` twice from its directory. The delivered PDF was rendered and visually inspected after compilation. The root manifest records SHA-256 identities for the deliverable files, excluding the manifest itself.

B Identity of the new source artifacts

Original class-A source SHA-256: cddbed7f20e7a8b085b7ca0e76811d07b021522375643b5d1134eec0d2d23164
Specialized source SHA-256: 5c2ba0b336ea1e32921154a050c0231b5d1c45d440a7dd348ae39a57ef364f5d
Zero/positive invariant SHA-256: f454cf5c9403613e81d27213aebfab472853ae6f2e3e6fbd3e3ee3753cff5d96

The joint invariant file stores one 512-bit little-endian set per old source location. The smaller register-1 and register-6 category families are separate JSON artifacts. The checker regenerates all three and independently verifies closure. A saved status string is never substituted for replaying the underlying obligations.

References

[1] V. Reshetnikov, *ProveIt, incoming research packets*, pinned commit `c086b5adaa622800936576 8946d691dfc49a20bb`, `docs/incoming`. https://github.com/VladimirReshetnikov/ProveIt/tree/c086b5adaa6228009365768946d691dfc49a20bb/docs/incoming.

[2] *Unary power-cover theory and PA*, `pa_unary_power_cover.zip`, especially `sections/unary_core.tex`, `unary_arithmetic.tex`, and `upper_transport.tex`. Supplied at the preceding pinned snapshot; included in `references/`.

[3] The five U208 proposal archives dated 29 October 2026, listed in Section 2. Exact archive SHA-256 and Git blob identities are in `provenance/`.

[4] E. Glazer, *K215 halts iff ZFC is inconsistent (Metamath Zero, PA)*. https://github.com/elliotglazer/k215-pa. Accessed 29 October 2026.

[5] E. Glazer, *K230: a 230-state Turing machine for the consistency of ZFC*, section "The theory K." https://github.com/elliotglazer/k230-rocq/blob/main/README.md#the-theory-k. Accessed 29 October 2026.

[6] V. Reshetnikov, *K215 verification artifacts*, ProveIt. https://github.com/VladimirReshetnikov/ProveIt/tree/main/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215.

[7] J. (jeff87654) and contributors, *busy-beaver, ZF/m254*, branch `zf-254`. https://github.com/jeff87654/busy-beaver/tree/zf-254/ZF/m254.